

OBJEKTORIENTERET ANALYSE, DESIGN OG IMPLEMENTATION

8: Dynamic interactions and sequences

Jan H. Mikkelsen

Cyber og Computerteknologi

2. semester – KBH – F26

FROM LECTURE 7

- We had the “what if two threads are waiting for the same object/resource” question yesterday
- Two scenarios
 - Both threads are waiting for the same condition
 - Both threads are waiting for different conditions

FROM LECTURE 7

- Both threads are waiting for the same condition
- Here a notify() in the balance-handler is technically enough but it is risky
- A depositing thread calls notify() and happens to wake the right thread — fine
- If there are other threads waiting for different conditions on the same object, notify() might wake the wrong one and the actual signal is lost

```
//Thread A and Thread B both waiting for balance > 0
synchronized(account) {
    while (balance <= 0) {
        account.wait();
    }
    withdraw();
}
```

FROM LECTURE 7

- Threads waiting for different conditions. A as before but B deposits
- Consider this sequence:
 - Thread A waiting - wants to withdraw
 - Thread B waiting - wants to deposit
- Producer deposits money and calls notify(), which wakes Thread B arbitrarily
- Thread B checks condition (balance >= maximum) and goes back to wait()
- Thread A is never woken, and it sits waiting forever

```
//Thread B waiting for balance < maximum  
synchronized(account) {  
    while (balance >= maximum) {  
        account.wait();  
    }  
    deposit();  
}
```

FROM LECTURE 7

- If multiple threads are waiting, then the `notifyAll()` option is needed to ensure correct execution always
- It comes with a price as waking threads unnecessarily causes:
 - Extra context switching
 - Lock contention as all threads must reenter competition to reacquire
 - Threads immediately going back to sleep after finding condition(s) false
- Thundering heard problem with lots of activity that produce nothing useful
- For high performance this all matters, but generally speaking the `notifyAll()` approach is the right one to follow

```
synchronized(account) {  
    balance += amount;  
    notifyAll();  
}
```

FROM LECTURE 7

- At the core of the issue is that `notify()` picks one arbitrary thread from the wait queue – yes .. crazy as f**k!
- The JVM specification deliberately does not define which thread receives the signal - it could be random, it could be FIFO, it could be priority-based, there is simply no way of predicting or controlling which thread gets woken
- Legacy .. a sophisticated notification system that could target specific threads or specific conditions was considered too expensive
- A simple and fast approach (wake one arbitrary thread) was chosen as a pragmatic trade-off
- *"We'll give you a basic signalling mechanism. It's your responsibility as a programmer to use it correctly."*

FROM LECTURE 7

6

- The use of ReentrantLock() solves the issues with multiple condition objects
- Each condition here has its own wait queue and there is no risk of waking the wrong thread
- No thundering herd, no need for notifyAll()
- This is the preferred approach over raw wait()/notify()
- Plain wait()/notify() is not adequate for complex scenarios

```
Lock lock = new ReentrantLock();
Condition canWithdraw = lock.newCondition();
Condition canDeposit = lock.newCondition();

// Thread A waits on specific condition
canWithdraw.await();

// Thread B waits on different condition
canDeposit.await();

// Producer wakes only the right threads
canWithdraw.signal(); // only wakes withdrawal waiters
canDeposit.signal(); // only wakes deposit waiters
```

Example code on Moodle as
CODE/ BankAccount.java

AGENDA

- Sequence diagrams
 - Synchronous and asynchronous interactions
- Design and pitfalls
 - Dos and don'ts
- Examples
- The important stuff
- Assignments



- To be able to use sequence diagrams for OO design
- To be able to understand and design for synchronous and asynchronous java programs
- To be able to design event driven programs

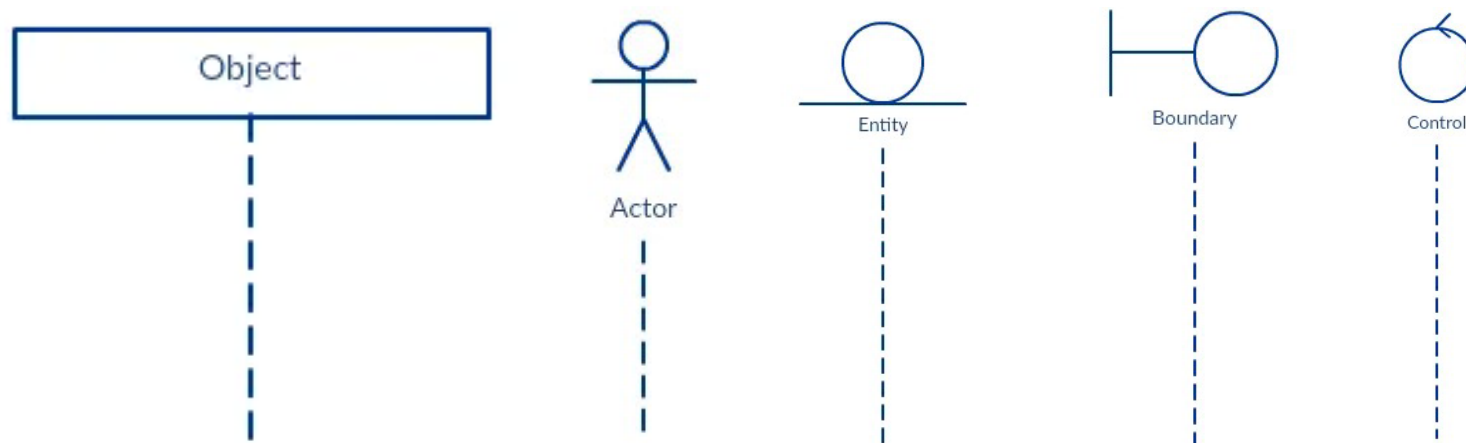
ENVIRONMENT DICTATES WHAT WE DO

- So far, all our design examples have been developed for a rather static environment
 - The world usually does not work like that
- We need to always adapt to our given environment, and we therefore need to consider
 - What has happened
 - What is happening now
 - What will happen next
- Writing code (designing systems) to cope with all and anything is not an option, but careful consideration can help us a lot
 - Here, we need to revisit our message sequence diagrams

MESSAGE SEQUENCE DIAGRAMS

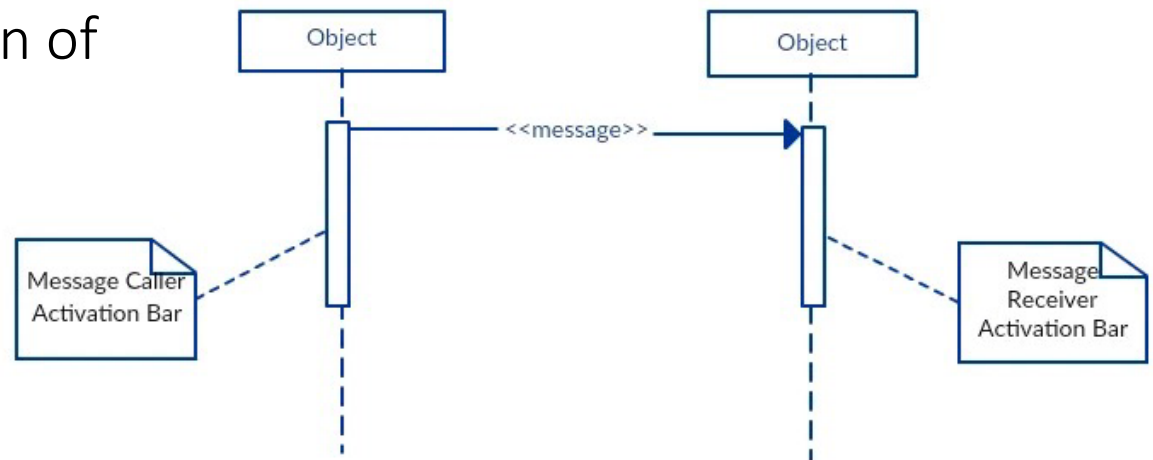
9

- Message sequence diagrams are useful in describing sequences of actions in time
 - Any action has a starting point – assumptions and conditions
 - Any action has a stopping point – when something has been achieved (or not)
 - Critical – timeline goes vertical and messages goes horizontal
- Specific notation as a part of UML



MESSAGE SEQUENCE DIAGRAMS

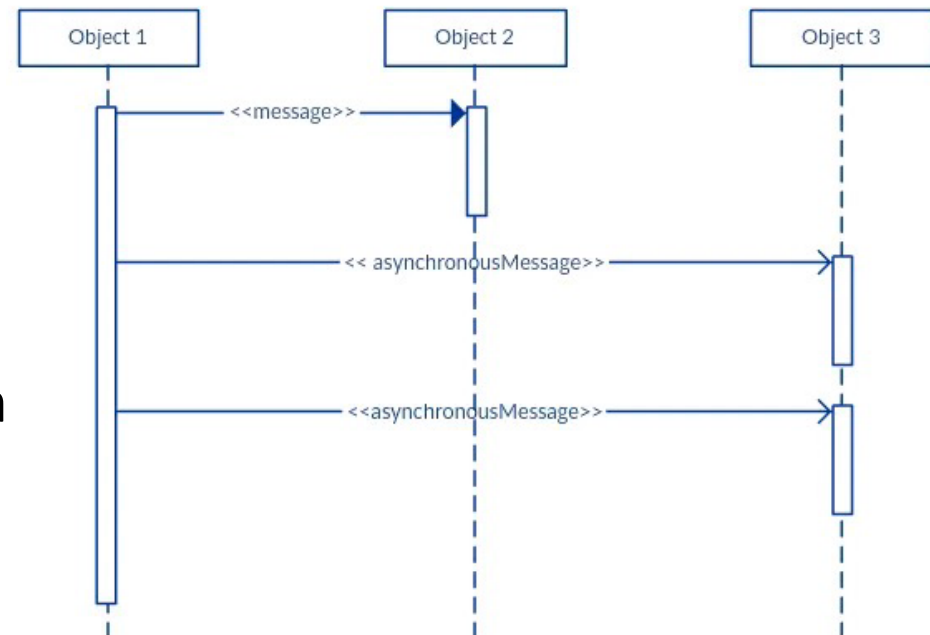
- Activation bars are, as mentioned last time, used to indicate that objects are active for a period of time
- Typically, a note on the activity is added
- Exchange of information between objects is shown by an arrow with text (indicating direction of information flow)
- The timed event can also be directly related to a method call



MESSAGE SEQUENCE DIAGRAMS

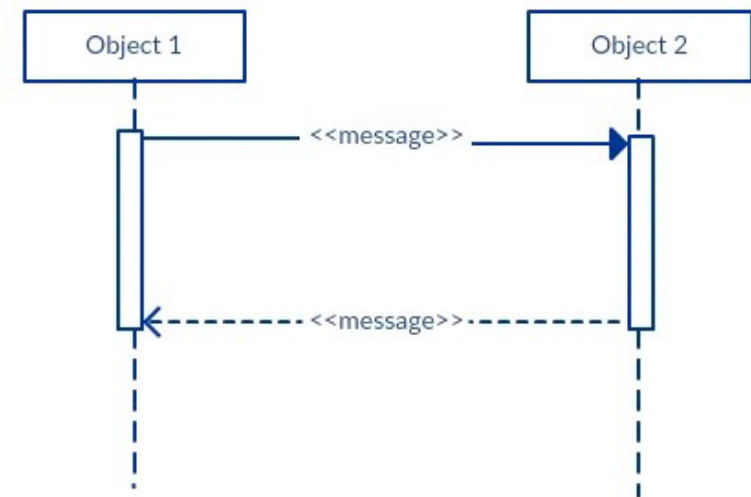
11

- Depending on whether the event is synchronous or asynchronous, different notations are used
- Synchronous
 - Thick arrow-head
- Asynchronous
 - Thin arrow-head
- Particularly for asynchronous messaging, the diagram shown is only one example of a realization



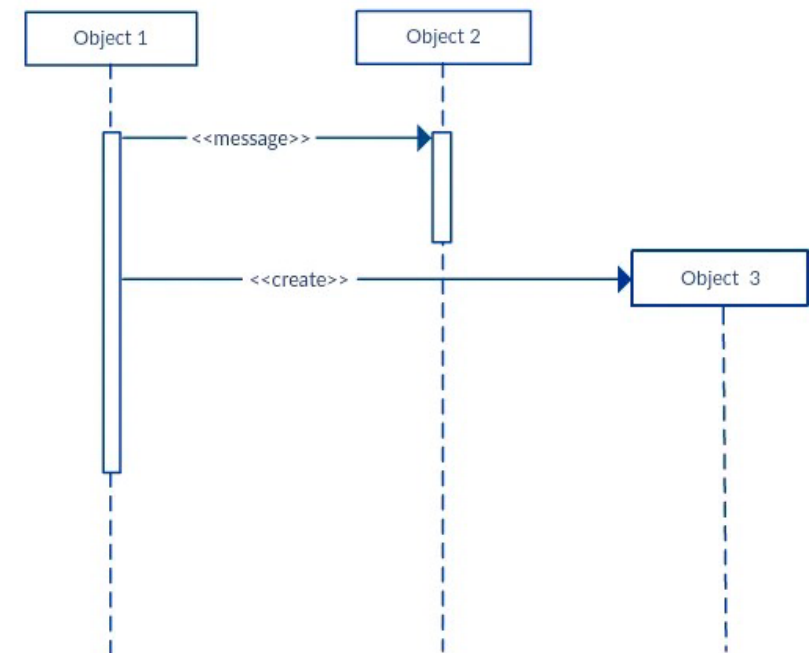
RETURN MESSAGES

- In many cases return messages are used to keep taps on things and to ensure that everything is running as planned
- However, return messages may be optional and often dotted lines are used to indicate this
- Using return messages are not always used as they tend to clutter the diagram
 - Do not use a double-arrow as this may lead to problems
 - If it is impossible to see what direction the main message flows then the whole point of the diagram is binned



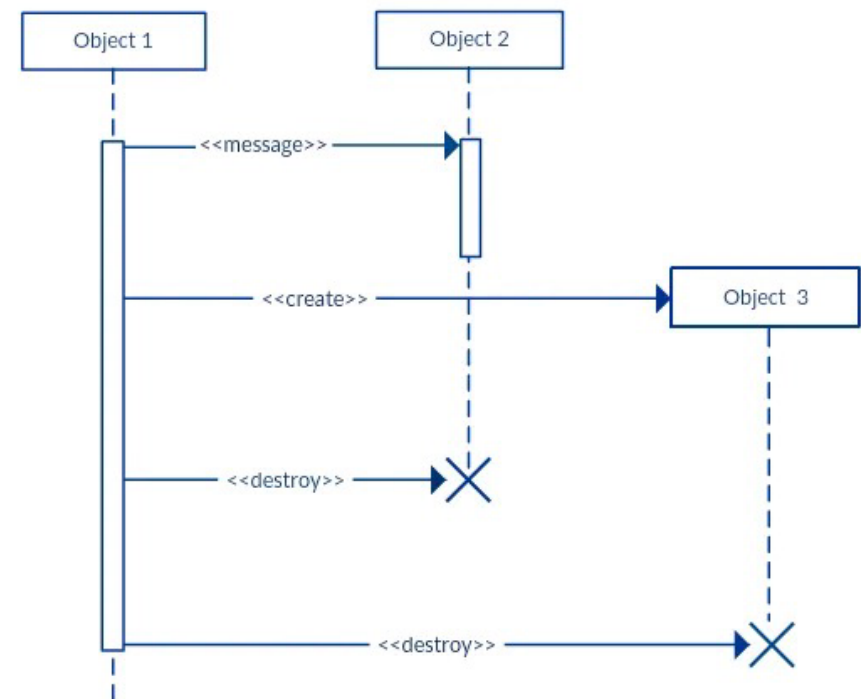
PARTICIPATION CREATION

- Sometimes it may be needed to create a participant on the fly
- This is indicated by dropping the participation further down the timeline, to indicate when the “add” takes place
- This is very often used in conjunction with a Thread being created and/or destroyed
- The whole process of keeping track of timing is – of course – especially important when working with multiple thread scenarios



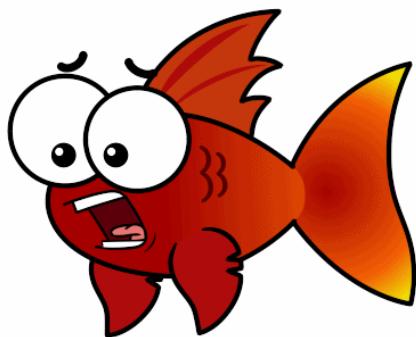
PARTICIPATION DESTRUCTION

- Keeping tabs on the destruction of participants is equally important
- Destruction of a participant is indicated by a large cross, which signifies termination of a life-line
- Such diagrams also help to identify problems, if e.g. the object is a Thread
- Destroying a thread, while another one is waiting for the Thread, can be difficult to spot in practice without a diagram like this
- It is all about establishing an overview of the tasks and events involved



BREAK – PAUSE(FISK)

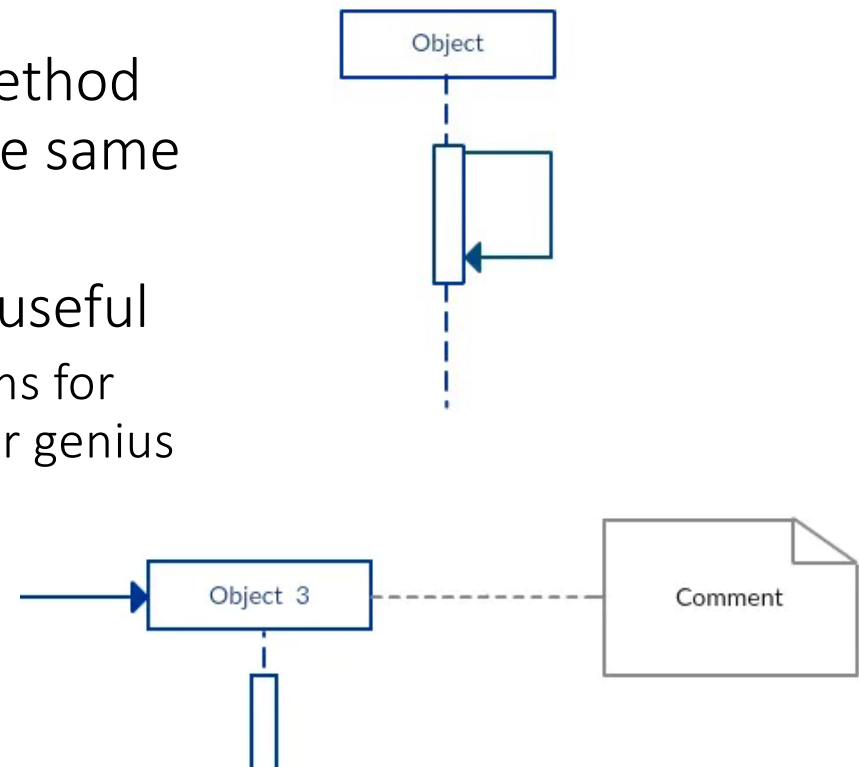
15



REFLEXIVE MESSAGES AND COMMENTING

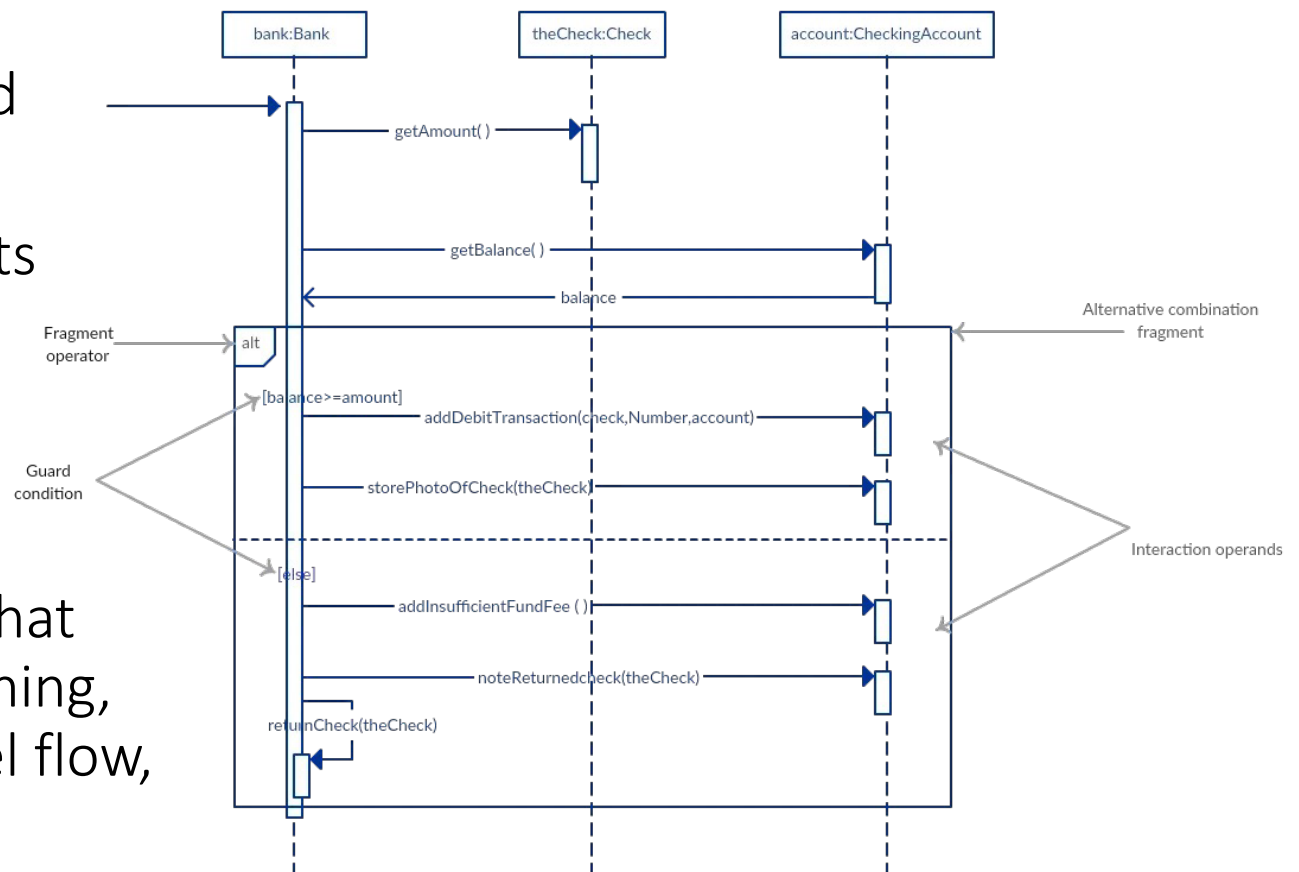
16

- In UML, a *reflexive message* (a *self-message* or *self-call*) is a message that an object sends to itself
- In Java terms, this corresponds to a method in a class calling another method of the same class, or calling itself recursively
- Comments in diagrams are extremely useful
 - Sometimes you will have to store diagrams for say 6 months. Do you still remember your genius stuff then?
 - Others may also not be able to read your mind, and determine why some object needs to do something specific



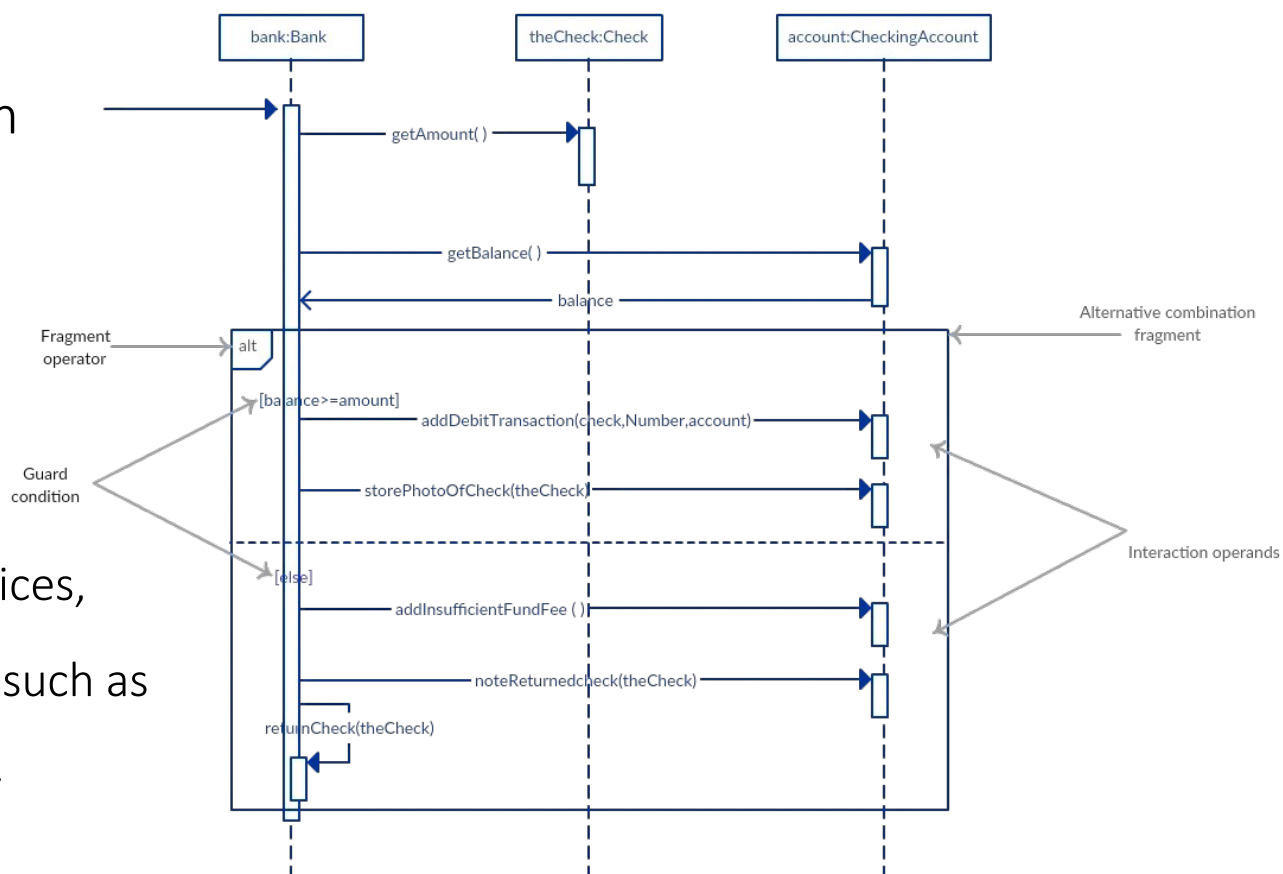
ALTERNATIVE EXECUTIONS AND FRAGMENTS

- Not all things happen as we planned, and we need to consider alternatives
- Using sequence fragments allows us to divide and structure diagrams accordingly
- A sequence fragment is a structured UML block that adds control logic (branching, loops, exceptions, parallel flow, etc.)



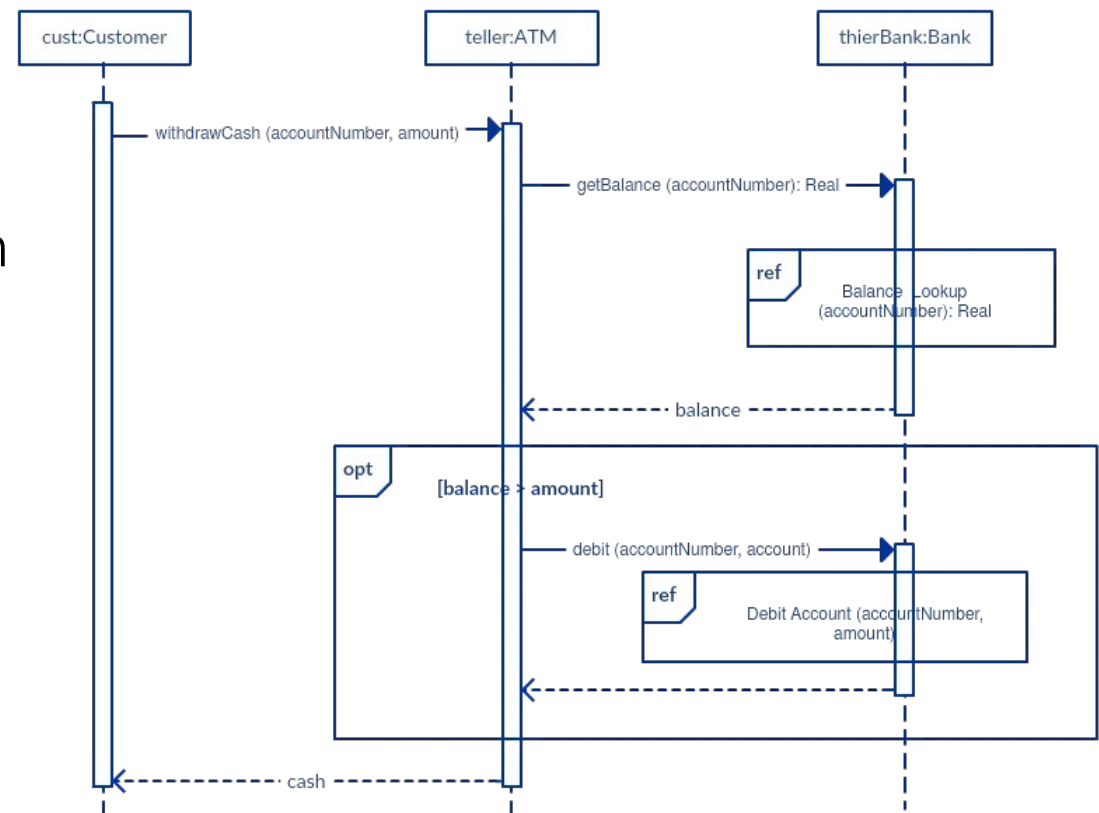
ALTERNATIVE EXECUTIONS AND FRAGMENTS

- An alt sequence fragment represents mutually exclusive paths of execution
- This is just like Java's if/else structure
- Only one operand can be executed at runtime depending on some set of parameters
- Used to model
 - control flow conditions, choices, loops, optional steps
 - multiple possible outcomes such as successful vs failed login
 - Auth rules, validation, state-dependent behaviour



IF TOO COMPLEX – USE REFERENCE FRAGMENTS

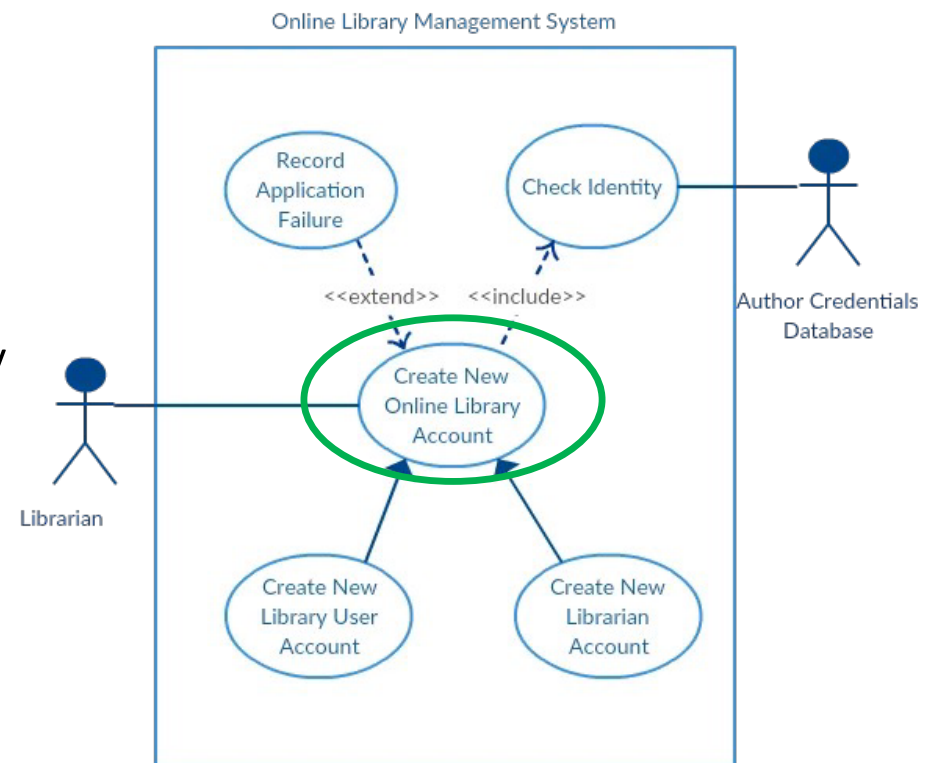
- Message sequence diagrams easily get too complex to use
- Using reference fragments to divide a complex diagram into several less complex is useful
- One can also reuse sequences in other diagrams
- It is not a competition of having the most “interesting” diagram
- The purpose is that we all understand what goes on in a specific scenario



SEQUENCE DIAGRAMS – HOW TO

20

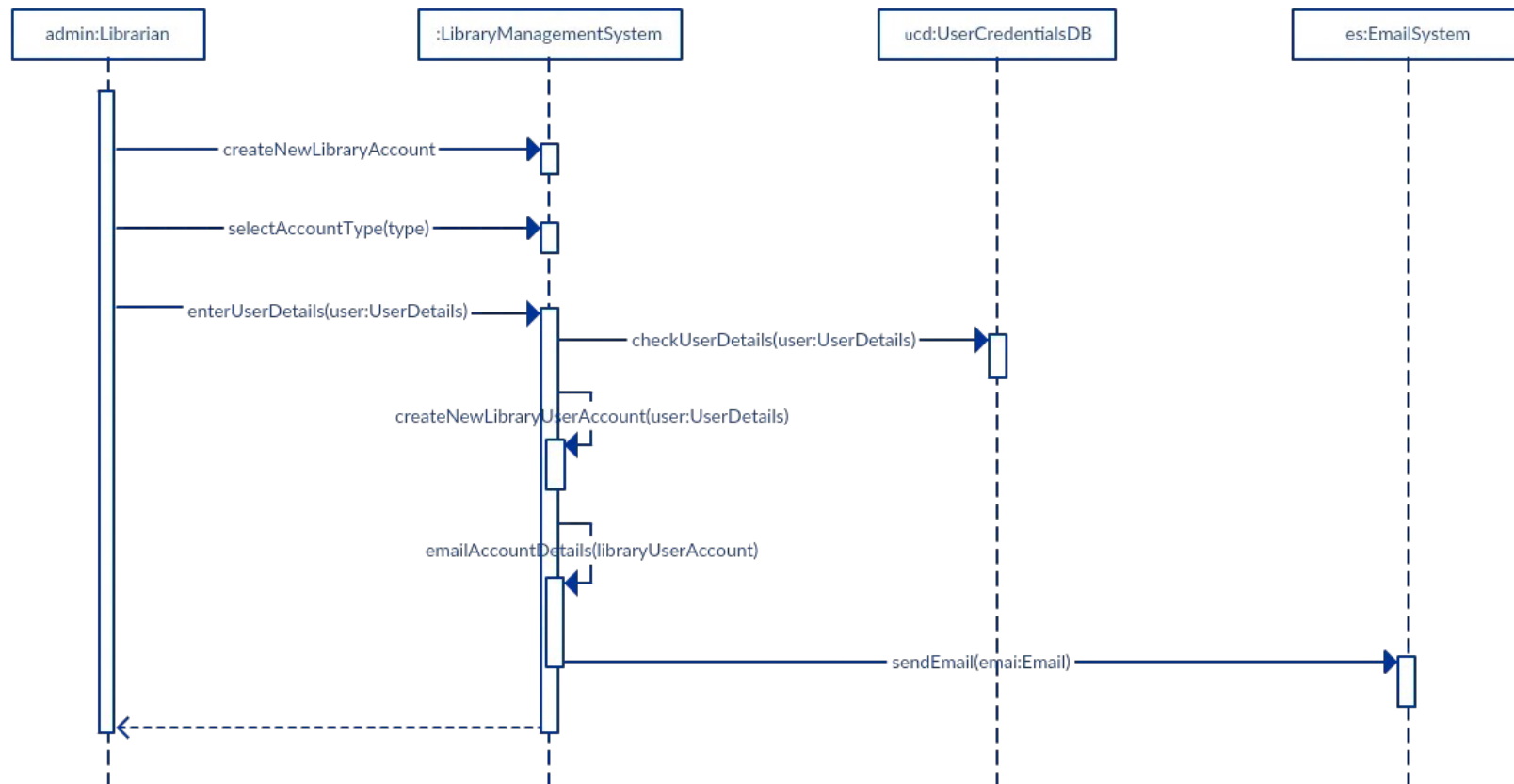
- To create a useful sequence diagram, we can step back to our use-case diagram
- A use-case description is already very close to the descriptive form of a message sequence diagram
 - This should not be a big surprise 😊
- Before proceeding we need to identify useful objects, e.g.
 - Librarian
 - Online Library Management system
 - User credentials database
 - Email system



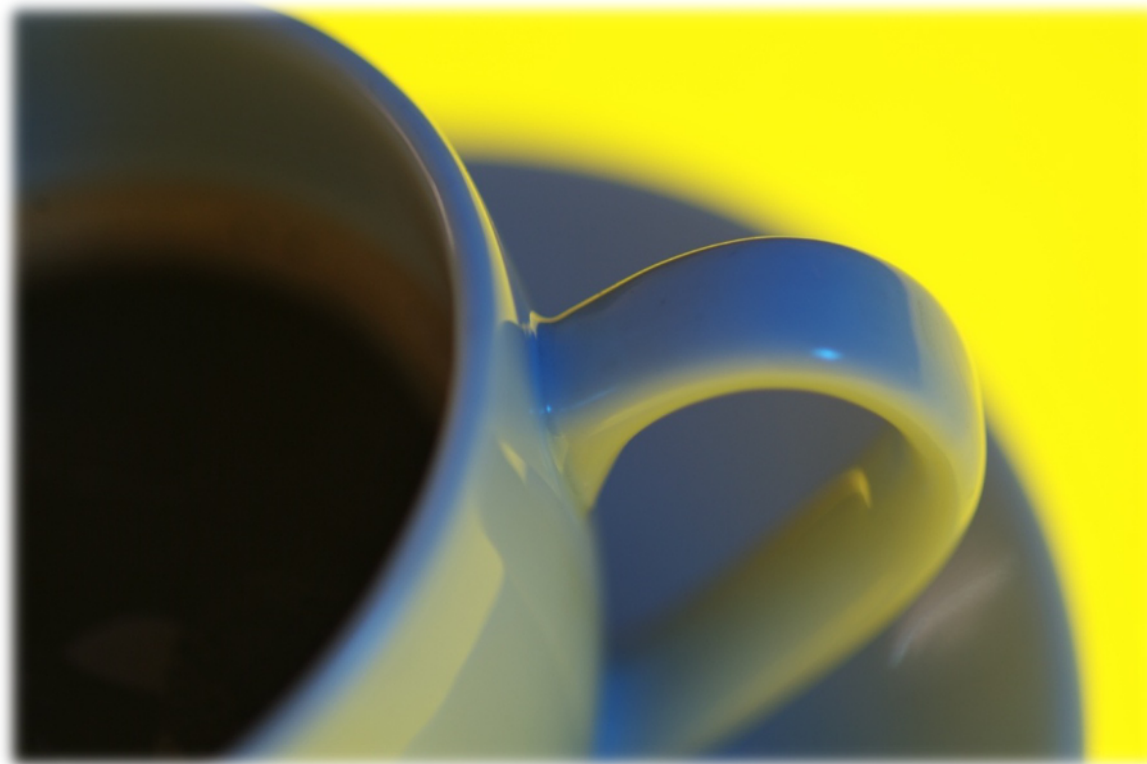
SEQUENCE DIAGRAMS – HOW TO

21

- As an example, let's consider how to create a new library-user account



"BIKER BREAK!"

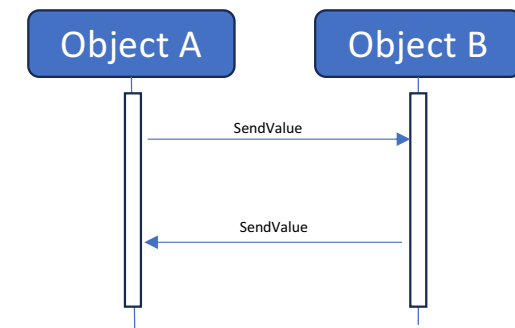
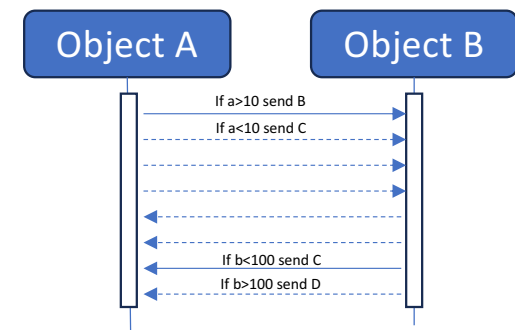


AGENDA

- Sequence diagrams
 - Synchronous and asynchronous interactions
- Design and pitfalls
 - Dos and don'ts
- Examples
- The important stuff
- Assignments

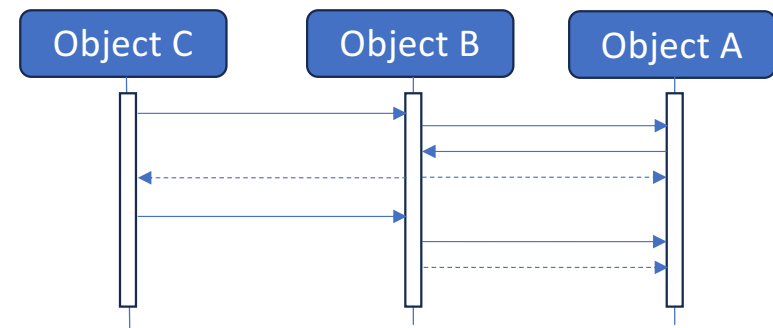
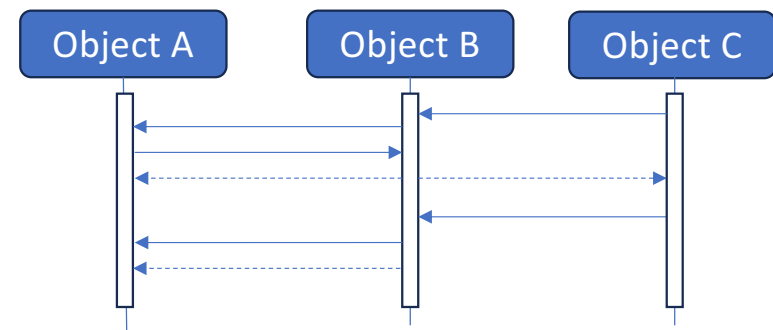
COMMON MISTAKES

- It is easy to fall into the too many details pit
- Remember that not every if-statement needs to be modelled in a diagram
- Instead try to attempt to simplify any message exchange to provide clarity at the same time
 - Abstractions are often more useful compared to complexity
 - Too much abstraction is also bad, so you need to aim for a balance here



COMMON MISTAKES

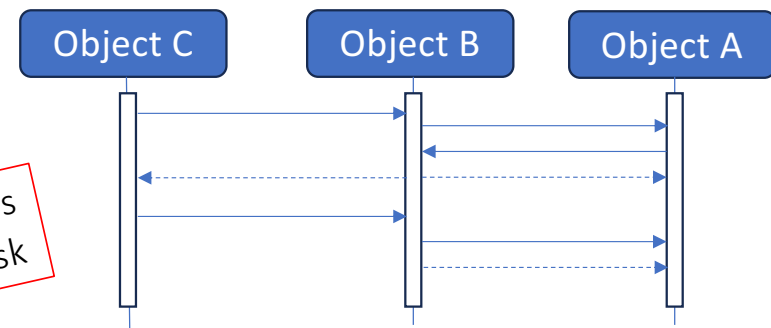
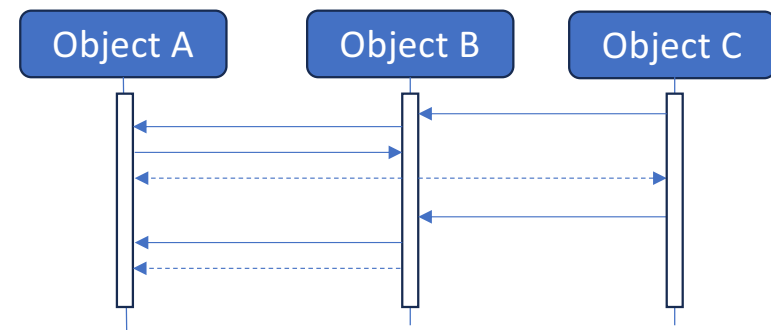
- To achieve a level of consistency, messages should (more often than not) run from left to right
 - This is simply back to the way western culture has a left→right reading style
- Although this is by no means critical, it may prevent recipients of the message sequence diagram from understanding the flow
- Or it could lead to misinterpretations, which of course also should be avoided



COMMON MISTAKES

- Obsolete sequence diagrams should be avoided at almost any cost, as they can be very dangerous and can lead to all kinds of misunderstandings
 - Obsolete – if code has changed and an early diagram no longer match the implementation
 - Which is then correct? The actual code or the diagram?
- If we cannot remember, we need to go through the code to identify which is, which (maybe it is neither)
- Time consuming!!

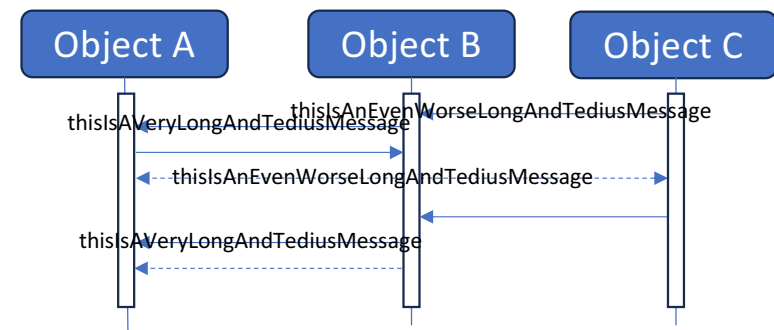
Any documentation or diagram that lives separately from the code carries this risk



COMMON MISTAKES

27

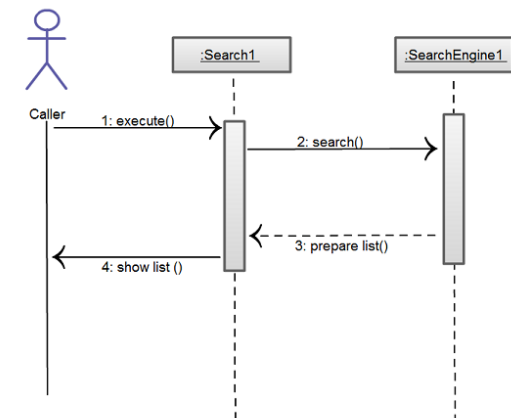
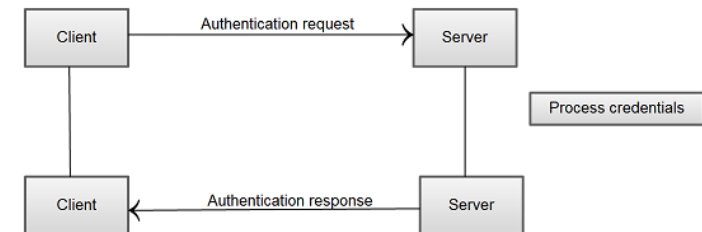
- Provide a visual trace between the text and message arrows
 - If you cannot read the message text, then understanding the sequence diagram can be a horrible pain
- Avoid sequence diagrams if you are dealing with simple logic
 - Yes, things can actually get too simple
 - The risk (one risk anyway) with small diagrams is that they easily get obsolete as nobody can be bothered to maintain the documentation
 - This way you risk having 100+ diagrams to update with a deadline tomorrow



COMMON MISTAKES

28

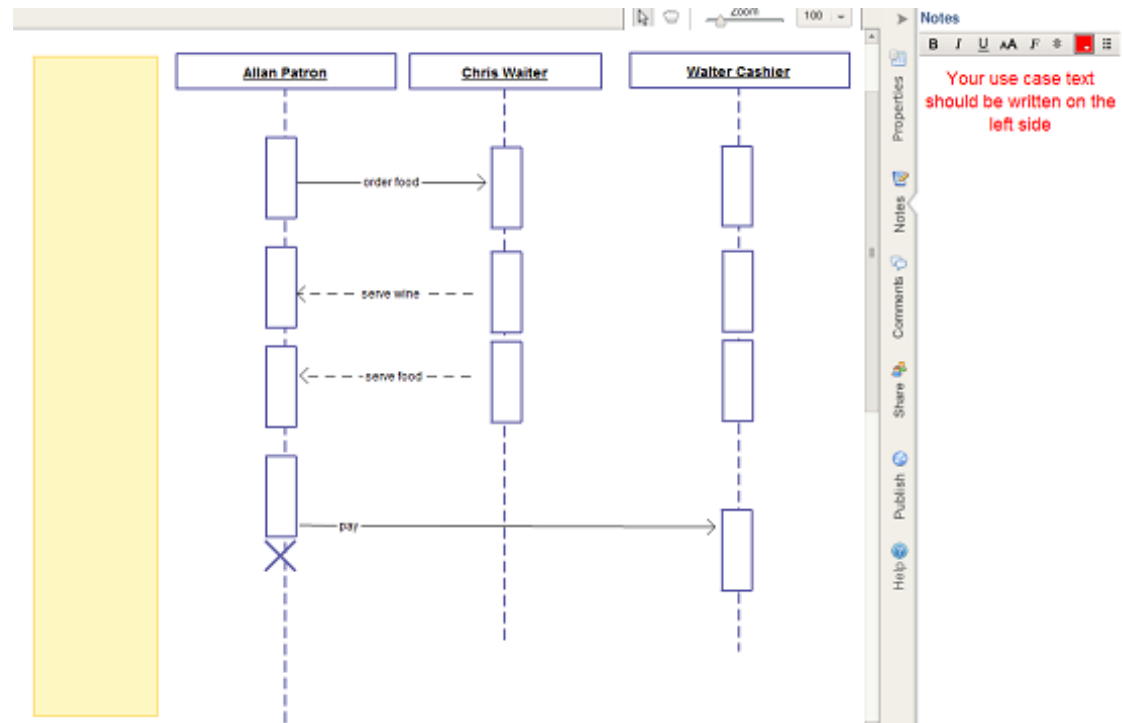
- Do not “plumb” your sequence diagram
 - Do not try to map every single method call and low-level detail
 - The diagram should have a special case with the authentication process
 - Later you can use it as a referred case that executes prior to your next case
 - Draw once and reference it by name in other diagrams
- Consider behaviour allocation
 - Which object is responsible for which action
- Do not get hung up on what you decide in the diagram
 - Diagrams are “thinking tools”, not contracts!
 - But keep diagrams updated!!



COMMON MISTAKES

29

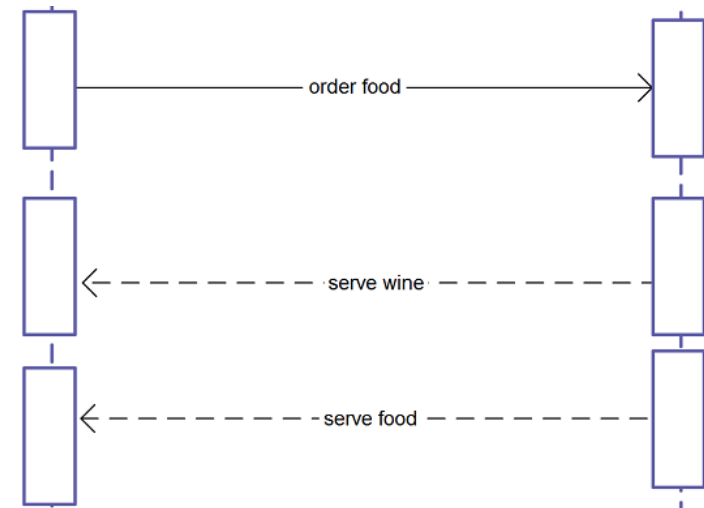
- Adding text to support the case is useful
- But be aware that text is following the sequences
- Not doing this, can really mess up the understanding of the diagrams
- Critical: these diagrams are here to help us understand the code and how it works over time
 - Not to confuse the "enemy", who might be your future self 😊



COMMON MISTAKES

30

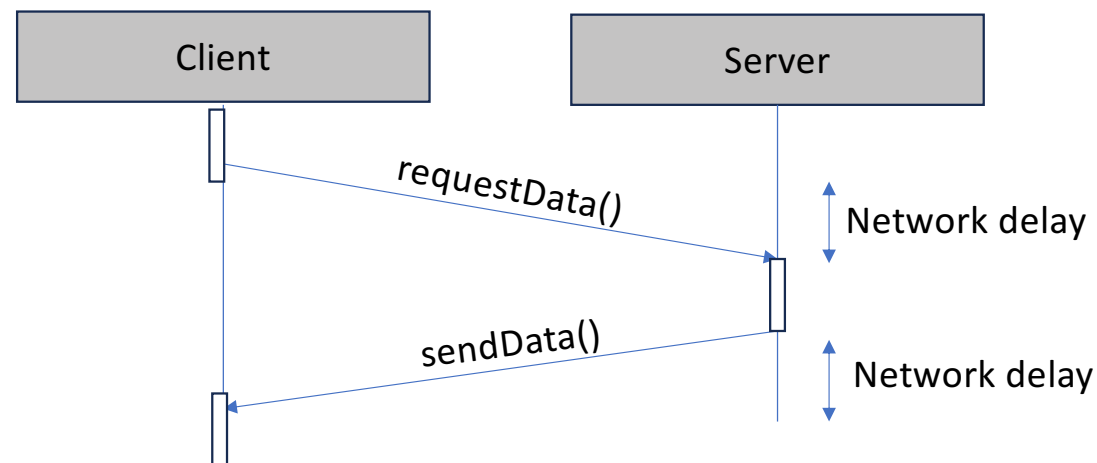
- Keep it basic and consider at all time, if methods are suitable for that particular object
 - Resist temptation to over-engineer
 - Responsibility-driven design, where every method should logically belong to the object it sits on
- Keep in mind you are still in a design phase, so changes costs literally nothing
 - But it will potentially be very costly if you change it in a month after most code has been written
 - The diagram phase is your cheapest opportunity to make mistakes



Fail fast, fail cheap!

EXTENSIVE USE OF SEQUENCE DIAGRAMS

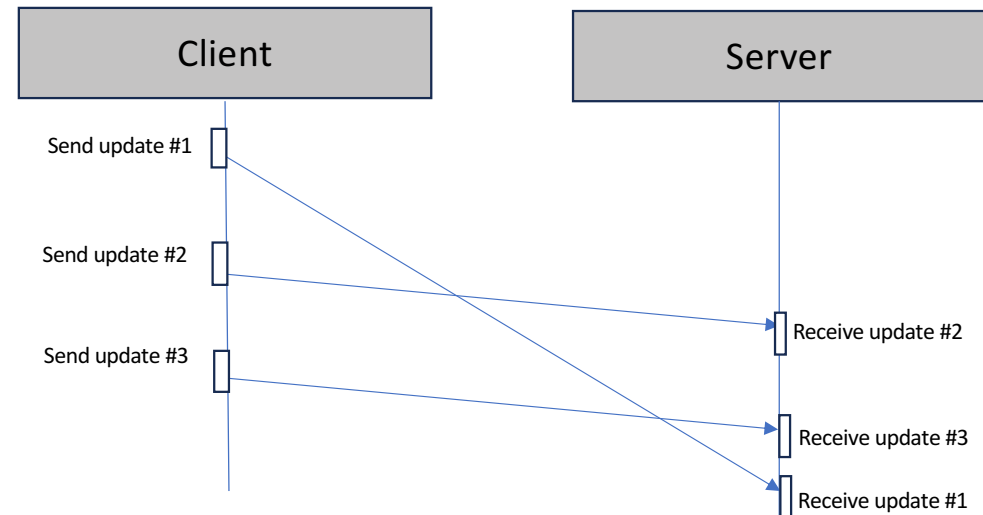
- Until now, we have seen message exchanges in complete horizontal lines
- If we have a network in between, or something that causes delays in other ways, we may (should!) indicate this in our MSD as well



EXTENSIVE USE OF SEQUENCE DIAGRAMS

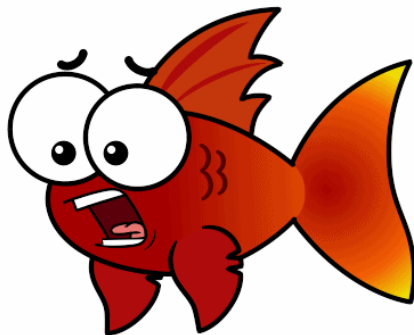
32

- It is useful to consider that a network (or any delay for that matter) may challenge the interaction between objects and components
- For example, detection of crossing User Datagram Protocol (UDP) packets (out-of-order receipt of data)
- UDP is a connection-less protocol that just fires packages off and crosses its fingers
 - You send package A and then B, but B arrives before A
- Not easy to detect in design phase – often just observed as some kind of strange fault in the system



BREAK – PAUSE(FISK)

33



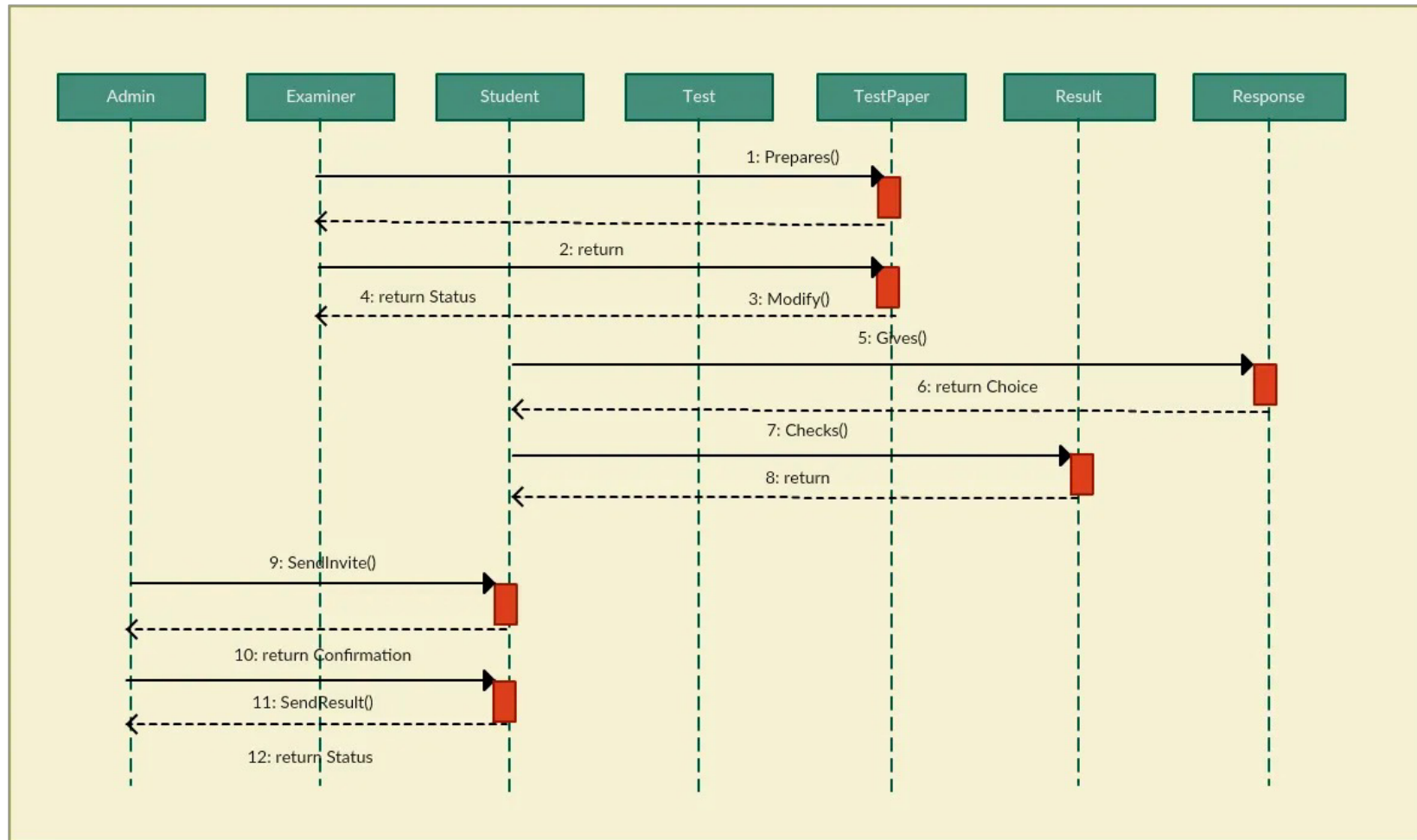
AGENDA

34

- Sequence diagrams
 - Synchronous and asynchronous interactions
- Design and pitfalls
 - Dos and don'ts
- Examples
- The important stuff
- Assignments

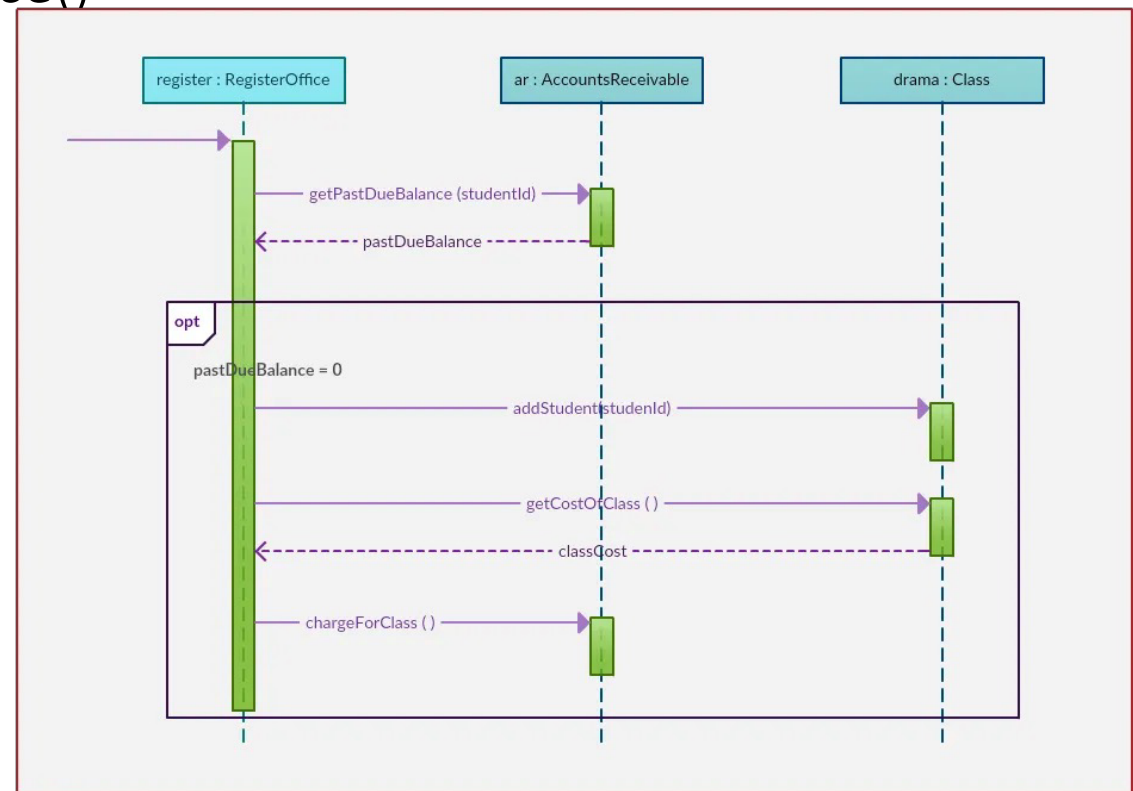
EXAMPLE DIAGRAM – AN ONLINE EXAMN SYSTEM

35



OPTION FOR COMBINATION FRAGMENT

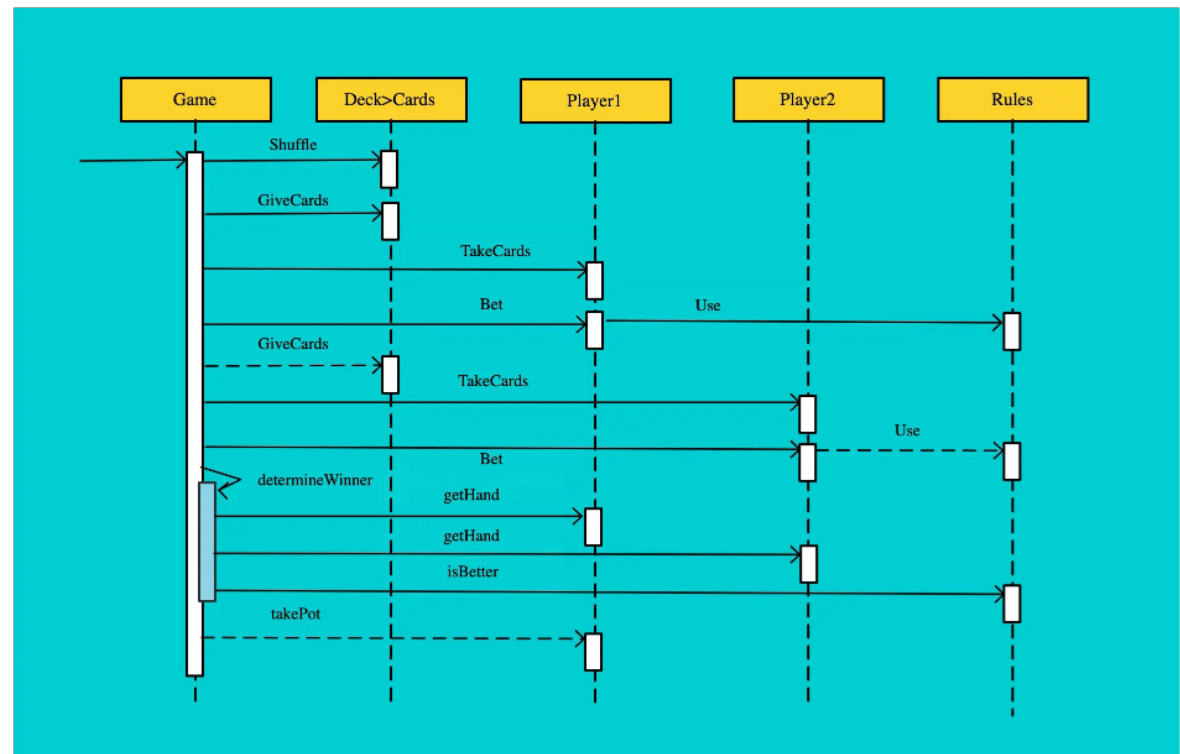
- Here, 'register' asks 'ar' for the student's past balance using the method call `getPastDueBalance()`
- Only if the `pastDueBalance` condition is true does the framed code execute
- It provides a clean way to model conditional behaviour
- It shows good responsibility allocation
 - AccountsReceivable owns the balance check
 - Class owns the enrolment



EXAMPLE DIAGRAM – A CARD GAME

37

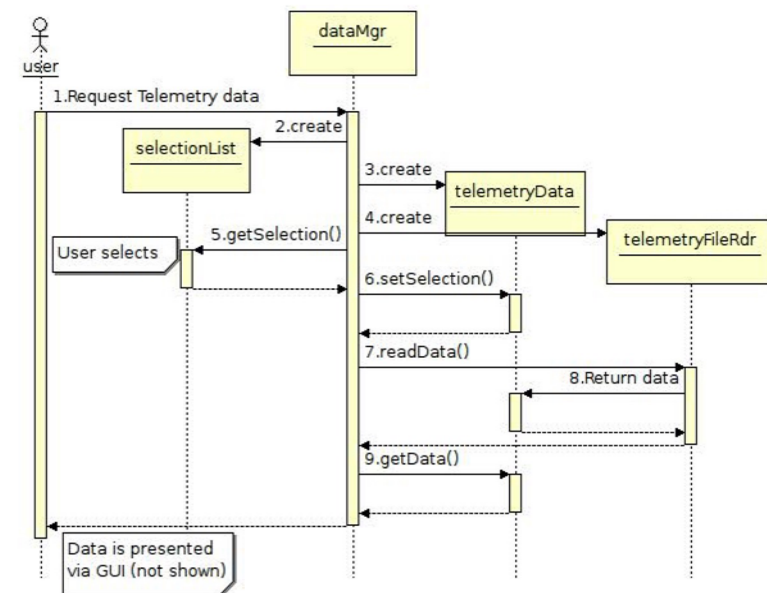
- Rules is a separate object — game logic is cleanly separated from players and the deck, which shows good responsibility allocation
- determineWinner is a self-call
 - Game is orchestrating its own winner logic rather than delegating
You could argue that this belongs on Rules
- There are no guard conditions or opt blocks
 - What happens if players tie? This is not modelled



EXAMPLE DIAGRAM – SATELLITE SYSTEM

38

- This has one notable difference as it includes a human actor
- The human actor is explicitly modelled, which makes it clear where human interaction fits in the flow
- The "not shown" note is honest and clean documentation — rather than overcomplicating the diagram it acknowledges the GUI exists elsewhere
- Sequence diagrams can/should show human interaction points as it makes it clear where the system is waiting



AGENDA

39

- Sequence diagrams
 - Synchronous and asynchronous interactions
- Design and pitfalls
 - Dos and don'ts
- Examples
- The important stuff
- Assignments

THE IMPORTANT STUFF

- Method References and Lambdas (::) enable clean behavioural modelling and links directly to sequence diagram interaction
 - Method referencing like `comparing(Train::getTotalLengthM)` is almost self-documenting in the same way a sequence diagram is
- Responsibility Allocation matters enormously
 - Every method should logically belong to the object it sits on
 - Sequence diagrams are your cheapest opportunity to spot misallocated responsibilities
- Combined Fragments model real conditional behaviour
- Design changes are cheap now, expensive later
 - The cost of change curve is brutal
- Diagrams are living documentation
 - Obsolete diagrams are actively dangerous — worse than no diagrams at all
 - Keep them updated, annotate them with text that follows the sequence, and keep them honest

AGENDA

- Sequence diagrams
 - Synchronous and asynchronous interactions
- Design and pitfalls
 - Dos and don'ts
- Examples
- The important stuff
- Assignments



ASSIGNMENTS

- 1) Create a message sequence diagram for the train search tool from last time
- 2) Add functionality (and implement this) to insert new trains into the list as well as modify and remove these (e.g. change the number and types of wagons connected to a train). Draw message sequence diagrams for these actions (just some examples – as they are useful for discussing)
- 3) Consider that the input part of your program (and related validation for input) is located on a client, and the list of trains are located on a remote server that maintains this table, how would your message sequence diagram appear in that case. Redraw as needed
- 4) Draw the message sequence diagram with multiple clients in mind. Consider that there is always a delay between sending a query and receiving it, as well as sending a return message. Also, consider that it takes a little bit of time to update the table. Can things go wrong? If so, how can you avoid it by design?